CLOUDMEDIC: Verified, Least-Privilege Troubleshooting of AWS Network and IAM Misconfigurations with Language Models and Formal Repair

Mithin Sagar
Department of Computer Science and Engineering
[TODO: institution]
mithinsagar@gmail.com

***Abstract*—Cloud support engineers spend much of their time on configuration tickets: an instance that cannot reach a database, a Lambda function that times out, an `AccessDenied` that "should not happen". Large language models (LLMs) can read such tickets, but their diagnoses are unverified and their fixes often widen access, for example by opening a security group to `0.0.0.0/0` or granting `s3:*`. We present CLOUDMEDIC, a hybrid troubleshooter in which a small local LLM only translates the ticket into a formal *intent*, while formal models of VPC reachability and IAM policy evaluation find every failing hop and a counterexample-guided MaxSMT search synthesises the fix that restores the intent with the least added privilege. To measure this we define *excess privilege*, the network flows and IAM grants a fix creates beyond both the intended and the pre-fix configuration, and release CLOUDMEDIC-BENCH, 310 validated fault-injection scenarios over 26 fault classes and 5 reference architectures, including 50 multi-fault cases in which each fault masks the other. On a stratified 154-scenario core set, CLOUDMEDIC with a 7B model produces a working fix with zero excess privilege and zero regression in PENDING of tickets, compared with PENDING for the same model prompted with an expert checklist and PENDING when that model also receives verifier feedback. We validate the IAM model against the AWS policy simulator (PENDING agreement) and the network model against live probes on AWS (PENDING). Everything runs on a laptop at zero cost.**

I. INTRODUCTION

Misconfiguration, not software defects, is a dominant cause of cloud access and connectivity problems, and it is the core workload of cloud support roles. A typical ticket is short and imprecise ("our app server can no longer reach the database since last night's deploy") while the configuration that explains it is large and layered: security groups (stateful), network ACLs (stateless, ordered), route tables, NAT and internet gateways, peering, private DNS, and for access problems a stack of identity policies, permissions boundaries, service control policies (SCPs), resource policies, VPC endpoint policies and KMS key policies, evaluated in a precise order.

LLMs are attractive for this task because tickets are natural language, and recent work applies them to incident root-cause analysis [1], [2]. Two problems remain. First, an LLM diagnosis is a guess: nothing checks it against the configuration. Second, and less studied, an LLM *fix* can be worse than the fault. A model that "fixes" a denied request by granting a service wildcard, or a timeout by opening a port to the internet, closes the ticket and opens a security incident. Support engineers are expected to resolve tickets without weakening the customer's security posture, so a useful assistant must be judged on whether its fix is *correct and minimal*, not only on whether it names the right resource.

AWS already uses automated reasoning to answer *questions* about configurations: Zelkova encodes IAM policies in SMT [3] and Tiros answers network reachability queries [4]. Network verification more broadly is mature [5], [6]. These tools need a formal query, which customers do not write, and they report violations rather than synthesising repairs. CLOUDMEDIC connects the two worlds: the LLM produces the formal query from the ticket, and formal reasoning does everything that must be right.

Contributions.

1) **Executable models** of VPC packet reachability and IAM authorisation for a documented subset of AWS semantics, with *full-path evaluation* that reports every failing hop, so faults masked by earlier faults remain visible (section IV).
2) **Least-privilege repair** by counterexample-guided MaxSMT over a typed edit language, with cost equal to the measured privilege a candidate adds, branch-and-bound over non-additive interactions, and a collateral-damage guard (section V).
3) **The excess-privilege metric** for evaluating configuration fixes, and CLOUDMEDIC-BENCH, a fault-injection benchmark whose labels hold by construction and whose every scenario passes four validity checks (section VI).
4) **An evaluation** against LLM-only, checklist-prompted and verifier-feedback baselines, ablations of the cost function and model size, and fidelity checks of the models against AWS itself (section VII).

Code, benchmark and all run logs are released. All experiments use open-weight models on a laptop (Apple M5, 16 GB).

II. Background and Related Work

LLMs for cloud incidents. Ahmed et al. evaluated LLMs for recommending root causes and mitigations for production incidents [1]; RCACopilot retrieves diagnostic information and asks an LLM to classify the root cause [2]. Benchmarks for AI agents in operations now exist: AIOpsLab [7], ITBench [8], OpenRCA [9] and RCAEval [10]. They mostly target microservice application failures observed through telemetry. CloudMedic-Bench targets the complementary setting of *infrastructure configuration* faults explained by the control-plane state, and scores fixes, not only localisation.

Automated reasoning about cloud configuration. Zelkova [3] and Tiros [4] answer IAM and network questions with SMT and are used in production AWS features. Batfish [6] and header space analysis [5] verify network configurations. Our models are far smaller in scope; their purpose is to provide verified obligations and a repair oracle in a closed loop with an LLM.

Verification and feedback for LLM output. Iterative self-refinement [11] and tool feedback improve LLM answers, and hallucination is a known failure mode [12]. We compare against this design directly: the *verifier-loop* baseline gives the LLM the same verifier's failing checks and lets it revise its fix twice.

Least privilege and mutation. The principle of least privilege [13] motivates our cost function. Our benchmark construction borrows from mutation testing [14]: faults are generated by operators applied to known-good artefacts.

III. Problem Formulation

A *snapshot* S is a JSON description of an account's VPCs, subnets, route tables, gateways, peerings, endpoints, security groups, network ACLs, compute (EC2, Lambda, RDS), private hosted zones, IAM roles and policies, SCPs and resource policies. An *intent* ι states what the customer expects to work: either a flow $(src, dst, proto, port)$, where dst may be a DNS name, or a request $(principal, action, resource)$. A ticket T describes a failing intent in natural language. Given (T, S) a system outputs ranked root causes (c_k, r_k), a fault class from a closed vocabulary of 26 classes and the misconfigured resource, and a fix: a list of edits in a typed language of 23 operations (adding a security-group rule, adding a NACL entry, adding a route, adding or replacing a policy statement, associating a hosted zone, and so on). Edits that name non-existent resources or are malformed are rejected and counted.

Access surface and excess privilege. For a snapshot S we define its access surface $A(S)$ over a finite universe fixed per scenario: all flows between compute endpoints and external hosts on a set of commonly attacked ports plus every listening port, and all (role, action, resource) triples over a 34-action catalogue (16 actions marked sensitive) and all modelled resources. A flow is in $A(S)$ if the network path permits it, whether or not a service listens, because an open path is exposure. Let S_h be the healthy configuration, S_b the broken one and S_f the result of applying a fix. Then

$$\mathrm{Excess}(S_f) = A(S_f) \setminus \big(A(S_h) \cup A(S_b) \cup \mathrm{need}(\iota)\big), \tag{1}$$

$$\mathrm{Regression}(S_f) = A(S_b) \setminus A(S_f). \tag{2}$$

Excess counts access that neither the intended nor the pre-fix configuration granted, so residue of the fault itself is not blamed on the fix. A fix is *safe* if it restores ι with empty excess and empty regression. Because the universe is finite, wildcard grants are under-counted, so reported excess is a lower bound.

IV. Formal Models

Network. A flow is evaluated hop by hop: DNS resolution (private zones resolve only from associated VPCs with `enableDnsSupport`); source security-group egress; source subnet NACL egress (skipped within one subnet); routing by longest-prefix match to `local`, an internet gateway (requires an attached IGW and a public IP; Lambda ENIs never have one), a NAT gateway (whose own subnet must route to an IGW, with its NACL checked in both directions) or a peering connection (must be active, with a return route in the peer VPC); destination NACL ingress; destination security-group ingress (CIDR or security-group reference); whether the service listens; and the return path. Security groups are stateful and skip the return path; NACLs are stateless, so the reply must be allowed for the client's whole ephemeral port range. We compute the fraction of that range a NACL allows: partial coverage produces an *intermittent* failure, a symptom customers report and LLMs rarely explain. Every hop is evaluated even after a failure (*full-path evaluation*), so one run exposes all faults on the path. Failures map to symptoms (timeout, connection refused, DNS failure, intermittent timeout).

IAM. Following the documented evaluation logic: any applicable explicit Deny (identity, boundary, SCP, resource policy, VPC endpoint policy) denies; every SCP must allow; an endpoint policy must allow requests through it; a permissions boundary must allow; then an identity Allow or a resource-policy Allow naming the principal (same account) or both (cross account). Reading or writing an SSE-KMS object adds a dependent `kms:Decrypt` or `kms:GenerateDataKey` request, which the key policy must grant directly or delegate to the account root, in which case an identity policy must also allow it. Conditions support the string, ARN, IP, Boolean and Null operators with `IfExists` and AWS's semantics for absent keys. When no identity policy allows a request, a near-miss analysis distinguishes a missing action, a resource-ARN mismatch, an unmet condition and an orphaned policy that would allow it if attached. Policy variables, set operators and session policies are out of scope.

V. CloudMedic

Intent extraction. The LLM sees the ticket and a compact inventory (names, IDs, IPs, roles, resource ARNs), not the full configuration, and returns an intent. The verifier checks it: an

Algorithm 1 Counterexample-guided least-privilege repair

$O \leftarrow$ failing checks of ι in S; $C \leftarrow \bigcup_{o \in O} \mathrm{cands}(o)$ with measured costs
$best \leftarrow \bot$; $B \leftarrow \emptyset$ ▷ blocked sets
loop
  $X \leftarrow \arg\min_{X \subseteq C} \sum_{x \in X} c(x)$ s.t. X covers O, exclusions, $X \notin B$ ▷ MaxSMT
  if unsat or $\sum_X c \geq \mathrm{cost}(best)$ **then return** $best$
  end if
  $S' \leftarrow \mathrm{apply}(S, X)$
  if ι holds in S' **then**
    $a \leftarrow$ true cost of X; update $best$
    if $a = \sum_X c$ and no access lost **then return** X
    end if
    $B \leftarrow B \cup \{X\}$
  else
    $O' \leftarrow$ new failing checks in S'; $O \leftarrow O \cup O'$; $C \leftarrow C \cup \mathrm{cands}(O')$
    if $O' = \emptyset$ **then** $B \leftarrow B \cup \{X\}$
    end if
  end if
end loop

intent that names unknown resources, or that *succeeds* in the snapshot (and so cannot be what the ticket reports as broken), is returned to the LLM with the reason, up to three attempts. When the model proposes several candidate intents, the one that fails with a diagnosable fault is kept.

Diagnosis. The failing checks of ι in path order, deduplicated by (class, resource), are the ranked root causes.

Repair (algorithm 1). Each failing check generates candidate edits at several breadths: the exact /32 source, the peer security group or subnet, the port from anywhere, all traffic; the exact action on the exact ARN, the object prefix, the whole service; an exception for the caller in a Deny, a condition corrected to the caller's actual context, or deleting the Deny. Each candidate is applied alone and its cost measured as $c = w_e + w_g|\Delta A| + w_x|\Delta A_{\mathrm{internet}}| + w_s|\Delta A_{\mathrm{sensitive}}| + w_l|\mathrm{lost}|$, plus its breadth rank to break ties toward the narrowest edit. Z3's optimising solver [15], [16] selects a minimum-cost set covering every failing check, subject to mutual exclusion between conflicting edits (two NACL entries with one rule number, two default routes). The set is applied and re-verified. Newly exposed failing checks, from faults that were masked, become obligations with their own candidates. Because added privilege is not additive (a boundary widening and an identity widening each add nothing alone but much together) the true cost of a verified set is recomputed; if it exceeds the solver's predicted cost the set is blocked and the search continues until the solver's lower bound reaches the best verified cost. Sets that remove any existing access are blocked.

VI. CloudMedic-Bench

Architectures. Five randomised templates: a three-tier web application (with strict or default NACLs), a serverless API (Lambda in private subnets, Aurora, an S3 gateway endpoint with an endpoint policy, a bucket policy that requires the endpoint, SSE-KMS), a multi-account data platform (permissions boundaries, a region-lock SCP, cross-account bucket and key grants, an IP-conditioned policy), peered shared-services VPCs with private DNS and a bastion, and queue workers (SQS queue policy, DynamoDB, Secrets Manager, a decoy unattached admin policy). Seeds vary IDs, CIDRs, ports, engines, names, NACL style and accounts.

Fault operators. Twenty-five operators read their targets from the healthy trace of an intent (the rule, route or statement that actually carried it), so they are template-independent: removing or mis-porting a security-group rule, inserting a NACL deny or deleting an allow, narrowing an ephemeral range, removing a default or peering route, detaching an IGW, losing a public IP, moving a Lambda into a public subnet, placing a NAT gateway in a private subnet, leaving a peering pending, disassociating a private zone, disabling DNS support, stopping a service, and for IAM removing an action, breaking a resource ARN (the classic bucket versus object ARN), adding an unmet condition, detaching a policy, adding identity, SCP or resource-policy denies, trimming or attaching a boundary, removing a cross-account grant, replacing a key policy and scoping an endpoint policy. Each operator also emits a reference fix in the edit language and a change-log line.

Validation. A scenario is kept only if (V1) the fault breaks the intent, (V2) the reference fix applies and restores it, (V3) the reference fix restores the exact healthy access surface, and for multi-fault scenarios (V4) each fault's fix alone leaves the intent broken. Of 2934 candidate faults on the test seeds, 47 failed V1; none failed V2 or V3; 0 pairs failed V4. Labels hold by construction; the evaluator's own diagnosis agrees with the label for 100.0% of the valid pool, and we used the disagreements found during development to correct two modelling gaps.

Tickets. Tickets are written as customers write them, with three levels: L1 (verbose error text including the reason phrase AWS returns, resource IDs, flow-log lines and the real change in the change log), L2 (terse error, names only, red-herring changes, the real change omitted or vague) and L3 (multi-fault, L2 style). Symptoms come from the simulated trace.

Splits. Dev (88 scenarios, seeds disjoint) was used to debug parsing; the test split has 310 scenarios (260 single-fault, 50 multi-fault). LLM baselines are expensive on a laptop, so all systems are compared on a stratified 154-scenario core set (four single-fault scenarios per class spread over templates and levels, plus every multi-fault scenario); CloudMedic and the non-LLM systems are also reported on the full split.

VII. Evaluation

Systems. *LLM only*: the model receives the ticket, the full snapshot, the class vocabulary and the edit language and returns root causes and a fix. *LLM + checklist*: the same plus an expert troubleshooting checklist and an instruction to prefer the narrowest fix. *LLM + verifier loop*: the checklist system whose fix is applied and checked against its own extracted intent by our verifier, with the failing checks fed back for up to two revisions. *Symbolic + template parser*: our engine with

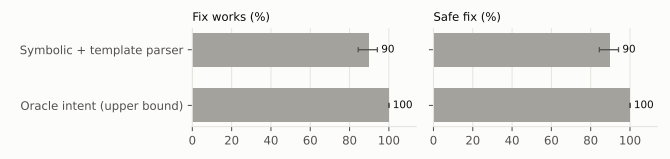


Fig. 1. Fix-works and safe-fix rates with 95% bootstrap intervals on the core set.

a regular-expression intent parser and no LLM. CLOUDMEDIC and CLOUDMEDIC*, no LP cost* (every edit costs the same; ties broken at random). *Oracle intent*: our engine given the gold intent, an upper bound that isolates intent extraction. Models: Qwen2.5-Instruct 7B and 3B [17], 4-bit, via MLX, greedy decoding, all completions cached and released.

Metrics. Exact intent; RCA@1 and RCA@3 (a gold class *and* resource among the top k; names are canonicalised for every system); fix works (gold intent holds after the fix); safe fix; over-privileged (works but excess > 0); invalid edit (any rejected edit); mean excess; seconds per ticket. We report 95% bootstrap intervals and exact McNemar tests on paired outcomes.

RQ1: Does verified diagnosis beat LLM diagnosis? Table I reports RCA@1 of **PENDING** for CLOUDMEDIC (7B) versus **PENDING** (LLM only) and **PENDING** (with checklist). [TODO: interpret once all runs finish: where each system fails, per family and level (breakdown table), McNemar p-values.]

RQ2: Are the fixes safe? CLOUDMEDIC fixes **PENDING** of tickets and **PENDING** safely; the strongest LLM baseline reaches **PENDING** safe fixes, and **PENDING** of the checklist baseline's tickets are closed with an over-privileged fix. [TODO: discuss typical over-privileged edits from excess_examples.]

RQ3: Which components matter? Removing the least-privilege cost (**PENDING** safe) isolates the repair objective; the 3B model (**PENDING** versus **PENDING**) tests whether the design lets smaller models compete; the oracle (100.0%) bounds the loss due to intent extraction (**PENDING** exact intents). [TODO: interpret.]

RQ4: Are the models faithful to AWS? The IAM model agrees with the AWS policy simulator on **PENDING** of **PENDING** random identity, boundary and resource-policy cases, and the network model with live probes on AWS in **PENDING** of **PENDING** probe groups covering stateful versus stateless filtering, rule order, partial ephemeral coverage, security-group references, refused versus timed-out connections and same-subnet NACL bypass. [TODO: discuss disagreements.]

VIII. THREATS TO VALIDITY

Synthetic tickets. Tickets come from templates; real tickets are messier. The template parser exploits their regularity, which is why it is a baseline and not our method; the LLM components never see a template. A study on real or human-written tickets is future work. **Circularity.** The same models that CLOUDMEDIC reasons with also simulate the symptoms. Labels do not depend on them (they come from the operators), and the fidelity experiments bound the gap between the models and AWS, but the oracle row is an upper bound of a system whose model is perfect, not of reality. **Scope.** The models cover a documented subset of AWS behaviour (no transit gateways, load balancers, IPv6, policy variables); faults outside it are invisible to CLOUDMEDIC but not necessarily to an LLM. **Metric.** Excess privilege is computed over a finite universe and is a lower bound; weights in the repair cost were set once and not tuned on test data. **Models.** One open model family at two sizes; larger or proprietary models may close part of the gap.

IX. CONCLUSION

Letting an LLM do only what it is good at, turning a ticket into a formal intent, and leaving diagnosis and repair to verified reasoning yields fixes that are both correct and minimal on a small local model. The excess-privilege metric makes "closed the ticket without weakening security" measurable, and we hope CLOUDMEDIC-BENCH encourages evaluating troubleshooting assistants on the safety of their fixes and not only on their answers.

REFERENCES

[1] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *International Conference on Software Engineering (ICSE)*, 2023.

[2] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *European Conference on Computer Systems (EuroSys)*, 2024.

[3] J. Backes, P. Bolignano, B. Cook, C. Dodge, A. Gacek, K. Luckow, N. Rungta, O. Tkachuk, and C. Varming, "Semantic-based automated reasoning for AWS access policies using SMT," in *Formal Methods in Computer Aided Design (FMCAD)*, 2018.

[4] J. Backes, S. Bayless, B. Cook, C. Dodge, A. Gacek, A. J. Hu, T. Kahsai, B. Kocik, E. Kotelnikov, J. Kukovec *et al.*, "Reachability analysis for AWS-based networks," in *Computer Aided Verification (CAV)*, 2019.

[5] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

[6] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015.

[7] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," *arXiv preprint arXiv:2501.06706*, 2025.

[8] S. Jha *et al.*, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," *arXiv preprint arXiv:2502.05352*, 2025.

[9] J. Xu *et al.*, "OpenRCA: Can large language models locate the root cause of software failures?" in *International Conference on Learning Representations (ICLR)*, 2025.

[10] L. Pham *et al.*, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proceedings of the ACM Web Conference*, 2025.

[11] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, 2023.

[13] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, 1975.

[14] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, 2011.

TABLE I
RESULTS ON THE 154-SCENARIO CORE SET (PERCENT UNLESS NOTED). GENERATED FROM THE RUN FILES.

System	Intent	RCA@1	RCA@3	Fix works	Safe fix	Over-priv.	Invalid edit	Excess	Sec.
LLM only (7B)	PENDING (0/154)								
LLM + checklist (7B)	PENDING (0/154)								
LLM + verifier loop (7B)	PENDING (0/154)								
Symbolic + template parser	89.6	89.6	89.6	89.6	89.6	0.0	0.0	0.00	0.1
CloudMedic, no LP cost (7B)	PENDING (0/154)								
CloudMedic (7B)	PENDING (32/154)								
LLM + checklist (3B)	PENDING (0/154)								
CloudMedic (3B)	PENDING (0/154)								
Oracle intent (upper bound)	100.0	100.0	100.0	100.0	100.0	0.0	0.0	0.00	0.1

[15] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008.

[16] N. Bjørner, A.-D. Phan, and L. Fleckenstein, "νZ: An optimizing SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2015.

[17] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.